

Project Title

**Electric Motor Temperature Prediction
Using Machine Learning**

Team ID : LTVIP2026TMIDS87983

TABLE OF CONTENTS

S.No	Main Section
1.	INTRODUCTION
2.	PROJECT DESCRIPTION
3.	TECHINICAL ARCHITECTURE
4.	PRE-REQUISITES
5.	PRIOR KNOWLEDGE
6.	PROJECT OBJECTIVES
7.	PROJECT STRUCTURE
8.	ADVANTAGES AND DISADVANTAGES
9.	CONCLUSION
10.	FUTURE SCOPE
11.	APPENDIX

1.INTRODUCTION

Project Title:

Electric Motor Temperature Prediction Using Machine Learning

Team ID : LTVIP2026TMIDS87983

Team Size : 4

Team Leader : Yadlapalli Lakshmi Prasuna

Team member : Addagalla Rekha Sree Lakshmi Hari Priya

Team member : Addala Keerthi

Team member : Ainapurapu L M D B Gopal

2.PROJECT DESCRIPTION

Electric motors are one of the most essential components in modern engineering systems. They are widely used in electric vehicles, industrial automation, robotics, manufacturing plants, aerospace systems, and renewable energy applications. Among various motor types, the Permanent Magnet Synchronous Motor (PMSM) is highly preferred due to its high efficiency, compact size, improved torque characteristics, and better power density. However, despite its advantages, temperature rise during motor operation remains one of the most critical challenges.

Temperature directly affects motor performance, efficiency, and lifespan. Excessive heat generation inside the motor can lead to insulation breakdown, degradation of winding materials, demagnetization of permanent magnets, and in severe cases, complete motor failure. Overheating not only reduces efficiency but also increases maintenance costs and system downtime. Therefore, accurate monitoring and prediction of motor temperature is crucial in ensuring reliability and operational safety.

Traditionally, temperature monitoring systems depend on physical temperature sensors embedded inside the motor structure. While sensors provide real-time temperature measurement, they have certain limitations. Sensors may fail in harsh operating conditions, may not be placed exactly at the hottest region of the motor, and most importantly, they do not provide predictive capability. In industrial environments, unexpected temperature spikes can occur due to sudden load variations, environmental changes, or electrical disturbances. Hence, there is a growing demand for intelligent systems that can predict motor temperature in advance rather than simply measuring it.

With the advancement of Artificial Intelligence and Machine Learning, predictive maintenance has become a powerful solution in engineering applications. Machine learning models can analyze historical motor operational data and identify complex nonlinear relationships between electrical parameters and temperature rise. By learning these relationships, a predictive model can estimate the Permanent Magnet (PM) surface temperature using measurable parameters such as ambient temperature, coolant temperature, motor speed, and stator currents.

This project focuses on developing a machine learning-based regression system to predict the PM surface temperature of a Permanent Magnet Synchronous Motor. The dataset used for this project is the PMSM Temperature Dataset obtained from Kaggle, which contains real operational motor data

collected under different working conditions. The dataset includes various electrical and thermal parameters recorded during motor operation.

The primary objective of this project is to build a reliable regression model capable of accurately predicting the PM surface temperature. The selected input features include ambient temperature, coolant temperature, d-axis voltage (u_d), q-axis voltage (u_q), motor speed, d-axis current (i_d), and q-axis current (i_q). The target variable for prediction is the Permanent Magnet surface temperature (pm).

To achieve accurate prediction, the project follows a structured machine learning pipeline. First, data cleaning and preprocessing are performed to handle missing values and remove irrelevant columns. Exploratory Data Analysis (EDA) is conducted to understand feature distribution, detect patterns, and analyze correlations between variables. Feature scaling is applied using MinMaxScaler to normalize the data and improve model performance.

Multiple regression algorithms are implemented and evaluated, including Linear Regression, Decision Tree Regressor, and Random Forest Regressor. These models are trained and tested using a train-test split strategy. Performance evaluation is conducted using standard regression metrics such as R^2 Score and Root Mean Square Error (RMSE). Among the implemented models, the Random Forest Regressor demonstrates superior performance with an R^2 score of approximately 0.89, indicating strong predictive capability.

After selecting the best-performing model, it is saved using joblib for deployment purposes. The trained model is then integrated into a Flask-based web application. The web application provides a simple and user-friendly interface where users can manually input motor parameters and receive real-time PM surface temperature predictions. This integration demonstrates the practical application of machine learning in real-world engineering systems.

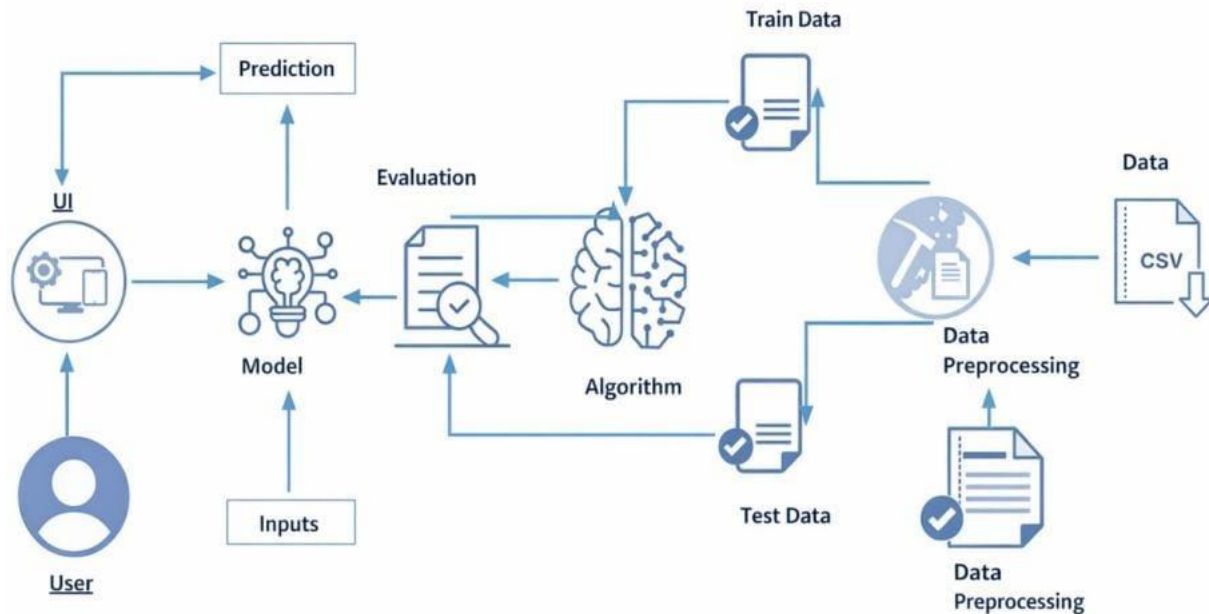
Furthermore, the project is deployed on a cloud platform, making the prediction system accessible online. The deployment process ensures that the model can be used remotely without requiring local setup. The complete project, including code and documentation, is maintained in a GitHub repository for version control and collaboration.

This project highlights the practical implementation of machine learning in electrical engineering and predictive maintenance systems. By accurately predicting motor temperature, the system can help reduce equipment failure, improve operational efficiency, and support intelligent monitoring

solutions. The methodology and framework developed in this project can be extended further by integrating real-time IoT sensor data, advanced deep learning techniques, and automated alert systems.

In conclusion, the Electric Motor Temperature Prediction project demonstrates how data-driven modeling techniques can enhance motor health monitoring and predictive maintenance strategies. It combines electrical engineering fundamentals with modern machine learning techniques to develop a scalable, deployable, and efficient predictive system.

3. TECHNICAL ARCHITECTURE



Technical Architecture

The above diagram illustrates the complete workflow of the Electric Motor Temperature Prediction system, starting from raw data collection to final user prediction. It represents the systematic flow of data through various stages of the machine learning pipeline.

Initially, the process begins with the dataset in CSV format, which contains motor operational parameters. This data undergoes preprocessing, where cleaning, feature selection, and scaling are performed to prepare it for model training. After preprocessing, the dataset is divided into training data and testing data. The training data is used by the machine learning algorithm to learn the relationship between input features and the target variable, while the testing data is used to evaluate model performance.

Once the algorithm is trained, the model is evaluated using appropriate regression metrics such as R^2 score and RMSE. After successful evaluation, the final model is saved and integrated into the web application.

On the user side, the user interacts with the system through the UI by providing input parameters such as ambient temperature, coolant temperature, motor speed, and electrical currents. These

inputs are passed to the trained model, which generates the predicted Permanent Magnet surface temperature. The prediction result is then displayed back to the user.

Thus, the diagram clearly demonstrates the end-to-end architecture of the system, showing how raw data is transformed into an intelligent predictive solution through machine learning and web integration.

4.PRE-REQUISITES

To successfully complete the Electric Motor Temperature Prediction project, certain software tools, programming knowledge, and Python libraries are required. These tools and packages are essential for data preprocessing, model building, visualization, and web deployment.

The following software and packages must be installed before starting the project implementation.

Software Requirements

1. Python

The primary programming language used in this project is Python (Version 3.8 or above). Python is used for:

- Data analysis
- Data preprocessing
- Model training
- Model evaluation
- Flask web integration

You can download Python from the official website:

<https://www.python.org/downloads/>

After installation, ensure that Python and pip are added to the system PATH.

2. Integrated Development Environment (IDE)

An IDE is required for writing and executing Python code. In this project, the following IDE can be used:

- **Visual Studio Code (VS Code)**
or
- **PyCharm**

VS Code is recommended because it is lightweight and supports Python extensions.

Download VS Code from:

<https://code.visualstudio.com/>

Install the Python extension in VS Code for better development experience.

3. Command Prompt / Terminal

The system terminal or command prompt is used to:

- Install required Python libraries
- Run the Flask application
- Execute Python scripts

Python Packages Required

To perform data analysis, machine learning modeling, and web integration, the following Python libraries are required.

Open Anaconda Prompt or Command Prompt as administrator and install the packages using the commands below:

1.NumPy

NumPy is used for numerical computations and array operations. It is essential for handling numerical data and performing mathematical calculations.

Command:

```
pip install numpy
```

2.Pandas

Pandas is used for data manipulation and data analysis. It helps in reading CSV files, handling missing values, filtering data, and performing preprocessing operations.

Command:

```
pip install pandas
```

3.Scikit-learn

Scikit-learn is the core machine learning library used in this project. It provides regression algorithms, model evaluation metrics, preprocessing tools, and train-test splitting functions.

Command:

```
pip install scikit-learn
```

4.Matplotlib

Matplotlib is used for data visualization. It helps in plotting graphs such as histograms, scatter plots, and performance comparison charts during Exploratory Data Analysis (EDA).

Command:

```
pip install matplotlib
```

5.Seaborn

Seaborn is an advanced data visualization library built on top of matplotlib. It is used for creating heatmaps, distribution plots, and correlation analysis.

Command:

```
pip install seaborn
```

6. SciPy

SciPy is used for scientific and statistical computations. It supports mathematical operations and enhances numerical analysis capabilities.

Command:

```
pip install scipy
```

7. Joblib

Joblib is used to save and load the trained machine learning model. After training the Random Forest model, it is saved using joblib for later deployment in the Flask application.

Command:

```
pip install joblib
```

8. Flask

Flask is a lightweight Python web framework used to build the web application interface. It enables routing, form handling, and integration of the trained model with the user interface.

Command:

```
pip install Flask
```

Optional Tools

Although not mandatory, the following tools can enhance project development:

- Git (for version control)
- GitHub (for repository hosting)
- Render (for cloud deployment)

Hardware Requirements

The project can be executed on a standard system with:

- Minimum 4GB RAM
- Dual-core processor
- Internet connection (for dataset download and deployment)

5.PRIOR KNOWLEDGE

To successfully understand and implement the Electric Motor Temperature Prediction project, prior knowledge in certain technical domains is essential. This project combines concepts from electrical engineering, machine learning, data analysis, and web development. A basic understanding of the following topics is required.

1. Machine Learning Concepts

Since this project is based on predictive modeling, knowledge of fundamental machine learning concepts is necessary.

● Supervised Learning

The project uses supervised learning, where the model is trained using labeled data. In supervised learning, the dataset contains input features along with the corresponding target output. The model learns the relationship between input variables and the output variable.

In this project:

- Input → Motor operational parameters
- Output → Permanent Magnet (PM) surface temperature

● Regression Algorithms

This project involves predicting a continuous value (temperature). Therefore, understanding regression techniques is important.

The following regression algorithms are used:

- Linear Regression
- Decision Tree Regressor
- Random Forest Regressor

Basic understanding of how these models learn patterns from data and make predictions is required.

● Model Evaluation Metrics

To measure model performance, evaluation metrics are used. Knowledge of the following metrics is essential:

- R^2 Score (Coefficient of Determination)
- Root Mean Square Error (RMSE)

R^2 Score indicates how well the model explains the variation in the target variable.

RMSE measures the average error between predicted and actual values.

● Train-Test Split

Understanding the importance of dividing data into training and testing sets is important. The training data is used to train the model, and testing data is used to evaluate its performance on unseen data.

2. Data Preprocessing Techniques

Raw data cannot be directly used for model training. Knowledge of basic preprocessing techniques is required.

● Handling Missing Values

Understanding how to identify and treat missing values in datasets.

● Feature Selection

Selecting relevant features that influence the target variable.

● Feature Scaling

Scaling ensures that all features are within a similar range. In this project, MinMaxScaler is used to normalize input features.

3. Python Programming

Since the project is implemented using Python, basic knowledge of Python programming is necessary.

Required Python skills include:

- Variables and data types
- Lists and arrays
- Functions
- Importing libraries
- Working with CSV file

Understanding how to use libraries like pandas and numpy for data handling is important.

4. Data Visualization

Basic understanding of visualization libraries such as:

- Matplotlib
- Seaborn

These tools help in performing Exploratory Data Analysis (EDA) to understand patterns, correlations, and distributions within the dataset.

5. Flask Web Framework

The project includes integration of the machine learning model with a web interface. Therefore, basic knowledge of Flask is required.

Important Flask concepts include:

- Routing
- Handling POST requests
- Rendering HTML templates
- Integrating a saved model into backend

Understanding how frontend communicates with backend is beneficial.

6. Basic Electrical Engineering Concepts

Since the dataset is related to electric motors, fundamental knowledge of the following concepts is helpful:

- Permanent Magnet Synchronous Motor (PMSM)
- Motor speed
- Voltage and current components (d-axis and q-axis)
- Temperature effects on motor performance

Understanding these concepts helps in interpreting model results correctly.

6 .PROJECT OBJECTIVES

The primary objective of the Electric Motor Temperature Prediction project is to develop a machine learning-based system capable of accurately predicting the Permanent Magnet (PM) surface temperature of a Permanent Magnet Synchronous Motor (PMSM). The project focuses on integrating data science techniques with practical engineering applications.

By the end of this project, the following objectives are achieved:

- Understand the fundamental concepts and techniques used in supervised machine learning, especially regression models.
- Gain a strong understanding of working with real-world datasets, including data exploration and interpretation.
- Develop knowledge of data preprocessing techniques such as handling missing values, feature selection, and feature scaling.
- Learn how to compare multiple regression algorithms and select the best-performing model based on evaluation metrics.
- Understand how to evaluate machine learning models using R^2 Score and Root Mean Square Error (RMSE).
- Gain experience in saving and loading trained models using joblib for deployment purposes.
- Develop skills in integrating machine learning models into a Flask web application.
- Learn how to deploy a machine learning application for real-time prediction.

The project not only strengthens machine learning concepts but also demonstrates their practical application in electrical engineering and predictive maintenance systems.

Project Flow:

The Electric Motor Temperature Prediction system follows a structured workflow from data collection to final user prediction.

The overall project flow is as follows:

- The user interacts with the web interface and enters motor operational parameters such as ambient temperature, coolant temperature, motor speed, voltages, and currents.
- The entered input values are sent to the backend Flask application through an HTTP POST request.
- The backend loads the trained machine learning model and applies necessary preprocessing (such as scaling).
- The model analyzes the input data and predicts the Permanent Magnet surface temperature.
- The predicted temperature is displayed on the user interface in real time.

This structured workflow ensures seamless interaction between data processing, model prediction, and user interface.

Project Activities

To accomplish the objectives of this project, the following major activities are completed:

● Data Collection

- o Collect the PMSM Temperature Dataset from Kaggle.
- o Understand dataset structure and feature descriptions.
- o Identify input features and target variable.

● Visualizing and Analyzing Data

Exploratory Data Analysis (EDA) is performed to understand data behavior and relationships between variables.

● Univariate Analysis

Analyze individual feature distributions using histograms and plots.

● Bivariate Analysis

Study relationships between input variables and target temperature.

- Multivariate Analysis

Analyze correlations between multiple features using heatmaps.

- Descriptive Analysis

Use statistical summaries such as mean, standard deviation, minimum, and maximum values.

This step helps in identifying patterns and understanding how different parameters influence motor temperature.

- **Data Pre-processing**

Before training the model, the dataset is cleaned and transformed.

- Checking for null values

Identify and handle missing values if present.

- Dropping unnecessary columns

Remove irrelevant features that do not contribute to prediction.

- Feature Scaling

Apply MinMaxScaler to normalize feature values.

- Splitting data into training and testing sets

Divide dataset to evaluate model performance on unseen data.

- Since the dataset consists of numerical values, categorical encoding is not required.

- **Model Building**

- Import necessary machine learning libraries from scikit-learn.

- Initialize regression models:

Linear Regression

Decision Tree Regressor

Random Forest Regressor

- Train the model using training data.
- Test the model using testing data.
- Evaluate model performance using:

R^2 Score

RMSE

- Compare models and select the best-performing algorithm.
- Save the final trained model using joblib.

● Application Building

After model development, a web application is created to provide real-time prediction access.

- Create HTML files for user interface:

index.html

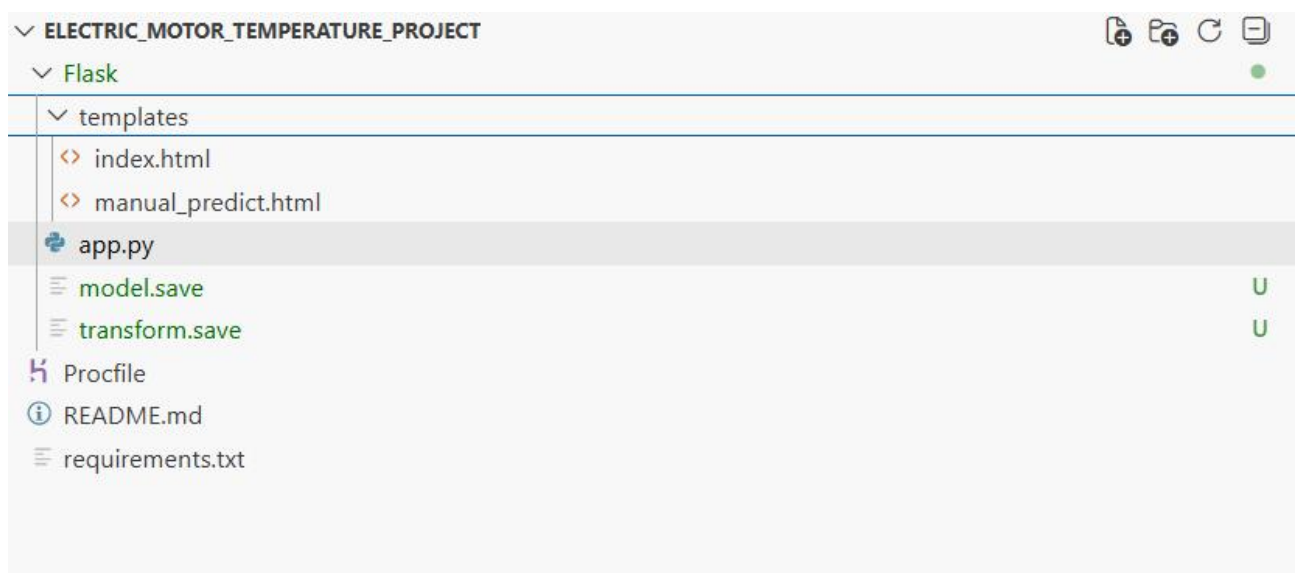
manual_predict.html

- Build Python backend code using Flask.
- Integrate saved model with Flask application.
- Handle form inputs and display prediction results.
- Test application locally and deploy online.

7 .PROJECT STRUCTURE

The Electric Motor Temperature Prediction project is organized in a structured folder format to ensure clarity, modularity, and ease of maintenance. Since the system is developed using Flask, the project follows the standard Flask application structure.

The main project folder contains the following files and directories:



- **app.py**

This is the main Flask application file. It contains:

- Flask app initialization
- Routing logic
- Model loading using joblib
- Input handling from HTML forms
- Prediction logic
- Rendering prediction results

The app.py file acts as the bridge between the frontend and the machine learning model.

- **model.save**

This file contains the trained Random Forest regression model.

After comparing multiple regression algorithms, the best-performing model is saved using joblib. This saved model is later loaded into the Flask application for real-time prediction.

- **transform.save**

This file stores the trained MinMaxScaler object.

Since scaling was applied during model training, the same scaler must be used during prediction to maintain consistency. This ensures that input values entered by the user are transformed exactly as during training.

- **templates Folder**

The templates folder contains the HTML files required for the Flask application. Flask automatically looks for HTML files inside this folder.

It includes:

- **index.html** – Home page
- **manual_predict.html** – Input form page for motor parameters

These pages allow user interaction with the prediction system.

- **requirements.txt**

This file lists all Python libraries required to run the application. It ensures that the project can be easily installed and executed on another system or cloud platform.

- **Procfile**

The Procfile is used for deployment purposes (for example, on Render). It specifies how the Flask application should be started in a production environment.

Milestone 1: Data Collection

Machine Learning models heavily depend on data. Data is the most crucial component that enables algorithms to learn patterns and relationships. The quality, quantity, and relevance of data directly influence the performance of the model. Therefore, collecting the correct dataset is the first and most important step in this project.

In this project, the dataset used is the **PMSM Temperature Dataset**, which contains real operational data of a Permanent Magnet Synchronous Motor (PMSM). The dataset includes electrical and environmental parameters recorded during motor operation under different conditions.

Proper installation of these prerequisites ensures smooth execution and successful completion of the project.

Activity 1: Download the Dataset

There are many open-source platforms available for collecting datasets such as:

- Kaggle
- UCI Machine Learning Repository
- Data.gov

For this project, the PMSM Temperature Dataset is downloaded from **Kaggle**.

Link:

<https://www.kaggle.com/wkirgsn/electric-motor-temperature>

The dataset is provided in CSV format and contains multiple motor-related features. The dataset includes measurements such as:

- Ambient temperature
- Coolant temperature
- d-axis voltage (u_d)

- q-axis voltage (u_q)
- Motor speed
- d-axis current (i_d)
- q-axis current (i_q)
- Permanent Magnet surface temperature (pm)

The target variable selected for prediction is:

pm (Permanent Magnet surface temperature)

After downloading, the dataset is stored in the project directory and used for further analysis and model development.

Milestone 2: Visualizing and Analysing the Data

Once the dataset is collected, the next step is to understand the data thoroughly. Raw data cannot be directly used for model building without proper analysis.

Exploratory Data Analysis (EDA) helps in:

- Understanding feature distributions
- Detecting patterns
- Identifying outliers
- Finding relationships between variables

Although there are many advanced techniques for analysis, in this project, essential visualization and statistical techniques are used.

Activity 1: Importing the Libraries

The necessary Python libraries are imported for data handling and visualization.


```

import numpy as np
import pandas as pd
from scipy import stats
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.svm import SVR
from sklearn.metrics import mean_squared_error, r2_score
import joblib
import warnings
warnings.filterwarnings('ignore')

df.head()

```

Activity 2: Reading the Dataset

Our dataset format might be in .csv, excel files, .txt, .json, etc. We can read the dataset with the help of pandas. In pandas we have a function called `read_csv()` to read the dataset. As a parameter we have to give the directory of csv file

```

df = pd.read_csv('pmsm_temperature_data.csv.csv')
df.head()

```

	u_q	coolant	stator_winding	u_d	stator_tooth	motor_speed	i_d	i_q	pm	stator_yoke	ambient	torque	profile_id
0	-0.450682	18.805172	19.086670	-0.350055	18.293219	0.002866	0.004419	0.000328	24.554214	18.316547	19.850691	0.187101	17.0
1	-0.325737	18.818571	19.092390	-0.305803	18.294807	0.000257	0.000606	-0.000785	24.538078	18.314955	19.850672	0.245417	17.0
2	-0.440864	18.828770	19.089380	-0.372503	18.294094	0.002355	0.001290	0.000386	24.544693	18.326307	19.850657	0.176615	17.0
3	-0.327026	18.835567	19.083031	-0.316199	18.292542	0.006105	0.000026	0.002046	24.554018	18.330833	19.850647	0.238303	17.0
4	-0.471150	18.857033	19.082525	-0.332272	18.291428	0.003133	-0.064317	0.037184	24.565397	18.326662	19.850639	0.208197	17.0

Activity 3: Univariate Analysis

Univariate analysis refers to the examination of a single variable at a time. The main objective is to understand the distribution, spread, central tendency, and presence of outliers in each feature.

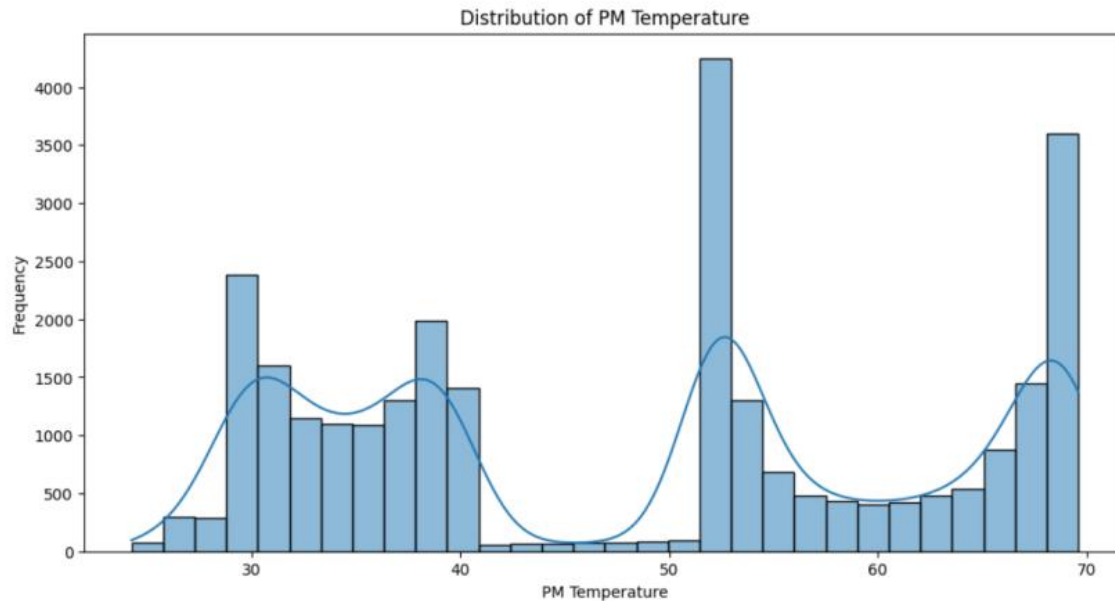
Since the PMSM dataset contains numerical variables such as ambient temperature, coolant temperature, motor speed, and Permanent Magnet (PM) surface temperature, distribution plots and boxplots are used.

Distribution Analysis

Histograms with Kernel Density Estimation (KDE) are plotted to observe the distribution of individual features.

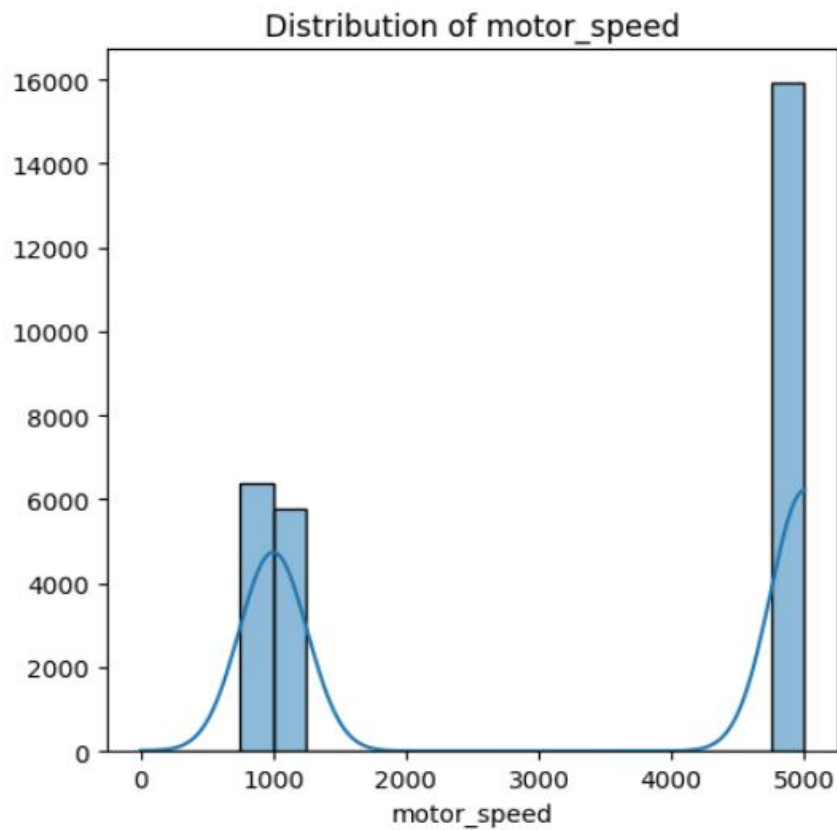
The distribution of PM temperature helps in understanding:

- Whether the data is normally distributed
- Presence of skewness
- Frequency concentration of temperature values



From the above distribution plot, it can be observed that PM temperature values are continuously distributed across a specific range. The data appears moderately spread without extreme abnormal spikes.

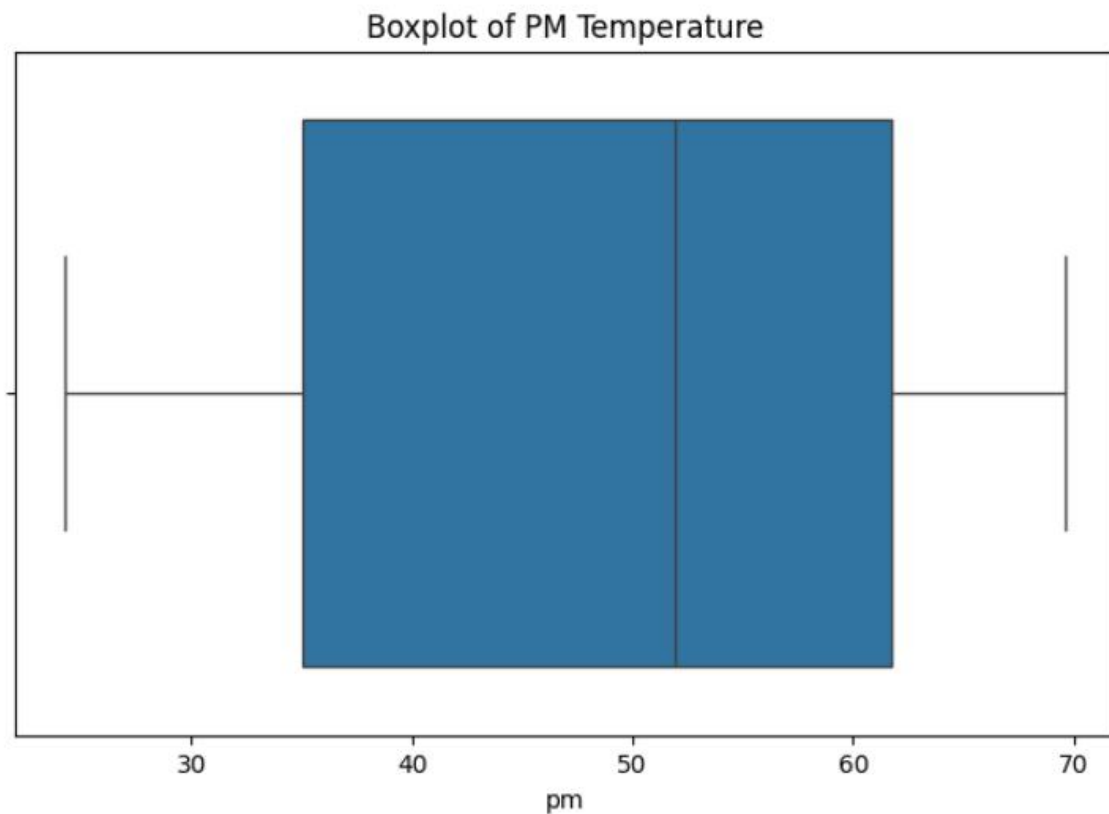
Motor speed distribution is also analyzed to understand operational range variations.



The motor speed distribution indicates variation across operating conditions, showing how speed changes impact thermal behavior.

Outlier Detection

- Boxplots are used to detect extreme values in the dataset.
- Boxplot of PM Temperature

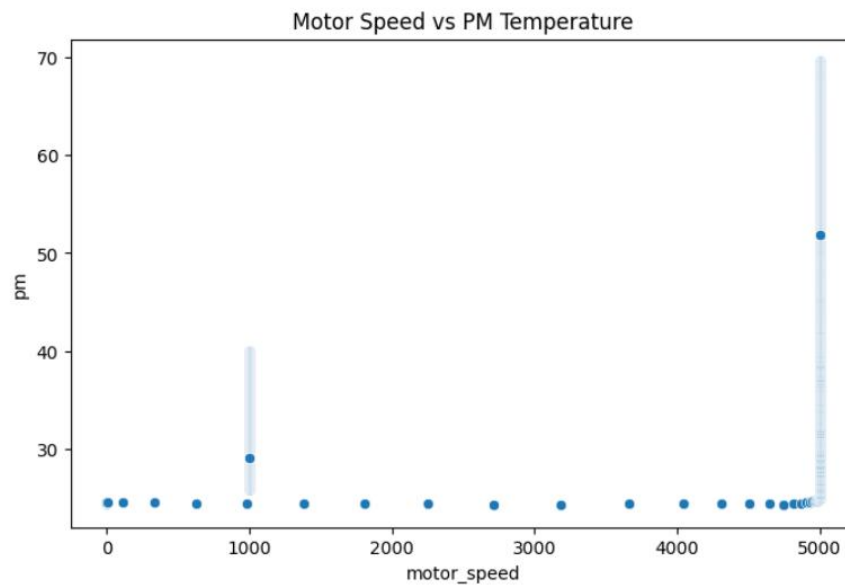


Activity 4: Bivariate Analysis

Bivariate analysis studies the relationship between two variables. Since the goal is to predict PM surface temperature, we analyze how each input feature influences the target variable.

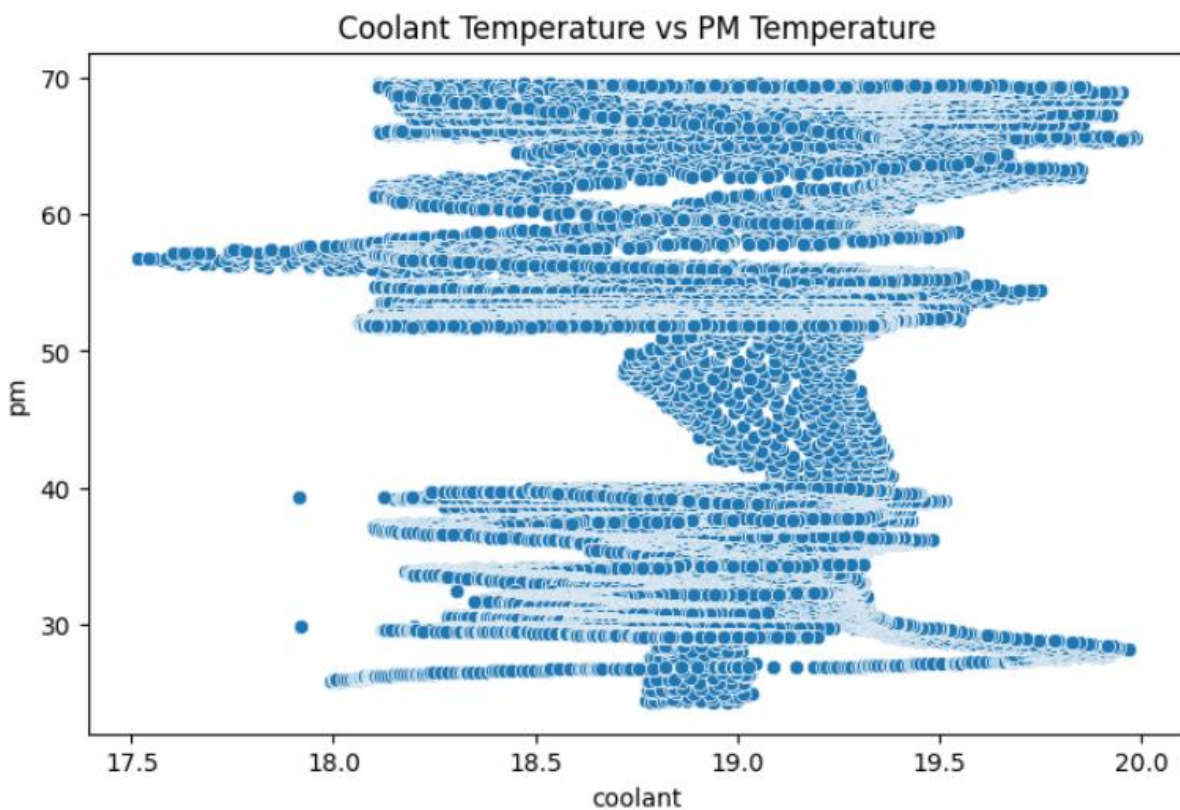
Scatter plots are used for visualization.

Motor Speed vs PM Temperature



The scatter plot shows the relationship between motor speed and PM surface temperature. It can be observed that as motor speed increases, the PM temperature generally tends to increase. This indicates a positive relationship between motor speed and temperature rise.

Coolant Temperature vs PM Temperature



The relationship between coolant temperature and PM temperature shows that higher coolant temperatures correspond to increased PM surface temperature. This confirms that environmental and cooling conditions significantly affect motor temperature.

Activity 5: Multivariate Analysis

Multivariate analysis examines the relationship between multiple variables simultaneously. It helps in understanding overall feature interaction and correlation.

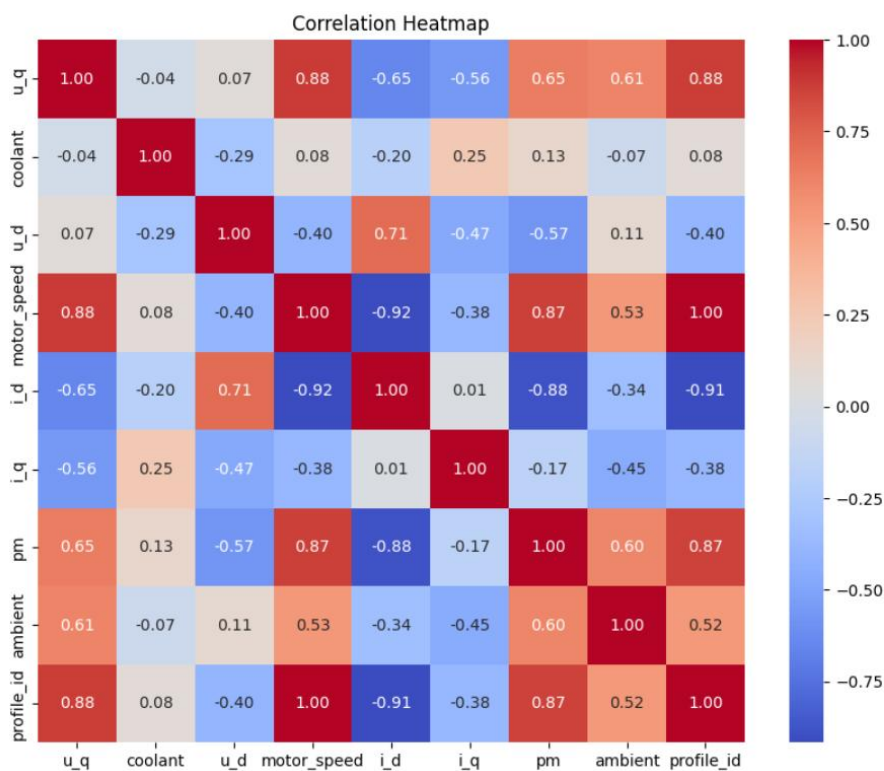
Correlation Heatmap:

A correlation matrix is generated to identify the strength of relationship between all numerical variables. The heatmap displays correlation coefficients between all features. From the heatmap, it can be observed that motor speed, coolant temperature, and current components show significant correlation with PM surface temperature. Strong positive correlation indicates that these variables are important predictors.

This step helps in feature selection and model building.

Pairplot Analysis

Pairplots are used to visualize relationships between multiple numerical features.



Activity 6: Descriptive Analysis

Descriptive analysis is used to summarize and understand the basic statistical properties of the dataset. It helps in identifying the central tendency, dispersion, and overall range of values present in each feature.

	u_q	coolant	u_d	motor_speed	i_d	i_q	pm	ambient	profile_id
count	28114.000000	28114.000000	28114.000000	28114.000000	28114.000000	28114.000000	28114.000000	28114.000000	28114.000000
mean	81.190443	18.896062	-40.321073	3265.553872	-77.710878	45.991534	48.281217	23.440903	11.813972
std	34.518994	0.426594	39.760383	1983.231479	51.983459	51.389945	14.200675	1.137857	5.944637
min	-0.974433	17.516184	-97.081703	-0.004142	-143.617004	-0.000785	24.264736	19.850620	5.000000
25%	49.389601	18.565468	-95.259256	999.991638	-138.160683	0.986889	35.070344	22.979434	5.000000
50%	87.026695	18.881069	-42.840477	4999.944824	-96.771873	53.120522	51.932928	23.335036	17.000000
75%	130.226151	19.270280	-6.150009	4999.949219	-43.510624	53.463409	61.754469	24.359284	17.000000
max	131.331100	19.986029	0.851181	4999.971191	0.004419	132.619171	69.588531	25.875355	17.000000

In this project, since the PMSM dataset consists entirely of numerical variables, descriptive statistical measures are used to analyze continuous features such as ambient temperature, coolant temperature, motor speed, current components, and Permanent Magnet (PM) surface temperature.

The `describe()` function from the pandas library is used to generate a statistical summary of the dataset.

Function Used:

`df.describe()`

Milestone 3: Data Pre-processing

After completing Exploratory Data Analysis, the next step is data preprocessing. Raw datasets cannot be directly used for training machine learning models because they may contain inconsistencies, irrelevant features, or values with different scales. Proper preprocessing ensures better model performance and reliability.

The PMSM Temperature Dataset is preprocessed through the following steps:

- Checking for missing values
- Dropping unnecessary columns
- Feature selection
- Feature scaling
- Splitting data into training and testing sets

Activity 1: Checking for null Values

The first step in preprocessing is to check whether the dataset contains any missing or null values.

Missing values can negatively affect model training and lead to incorrect predictions. Therefore, identifying and handling them is important.

The following functions are used:

df.shape

df.info()

df.isnull().sum()

```
<class 'pandas.core.frame.DataFrame'>
Index: 28114 entries, 0 to 28113
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  -
0   u_q              28114 non-null  float64
1   coolant          28114 non-null  float64
2   u_d              28114 non-null  float64
3   motor_speed      28114 non-null  float64
4   i_d              28114 non-null  float64
5   i_q              28114 non-null  float64
6   pm               28114 non-null  float64
7   ambient          28114 non-null  float64
8   profile_id       28114 non-null  float64
dtypes: float64(9)
memory usage: 2.1 MB
```

```
df.isnull().sum()
```

```

0
u_q      0
coolant   0
u_d      0
motor_speed  1
i_d       1
i_q       1
pm        1
ambient   1
profile_id  1
```

```
dtype: int64
```

Observation:

From the analysis, it can be observed that the dataset does not contain significant missing values. Therefore, no imputation techniques are required.

Activity 2: Dropping Unnecessary Columns

The original PMSM dataset contains additional temperature measurements such as:

- stator_yoke
- stator_tooth
- stator_winding
- torque

Since the objective of this project is to predict the **Permanent Magnet (pm) surface temperature**, these columns are not required.

Removing unnecessary columns improves model efficiency and reduces computational complexity.

```
df = df.drop(['stator_yoke', 'stator_tooth', 'stator_winding', 'torque'], axis=1)
```

This step ensures that only relevant input features are used.

Activity 3: Feature Selection

Feature selection is the process of selecting relevant input variables that influence the target variable.

Selected input features:

- ambient
- coolant
- u_d
- u_q
- motor_speed
- i_d
- i_q

Target variable:

pm

Separating features and target:

```
X = df[['ambient','coolant','u_d','u_q','motor_speed','i_d','i_q']]  
y = df['pm']
```

This separation is required before training the regression model.

Activity 4: Feature Scaling

Feature scaling is a crucial preprocessing step. Since different features have different ranges (for example, motor_speed values are much larger compared to ambient temperature), scaling ensures uniformity.

In this project, **MinMaxScaler** is used to normalize features between 0 and 1.

Why scaling is important:

- Prevents features with large values from dominating the model
- Improves convergence speed
- Enhances model accuracy

```
from sklearn.preprocessing import MinMaxScaler
```

```
scaler = MinMaxScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

Activity 5: Splitting Dataset into Training and Testing Sets

To evaluate model performance properly, the dataset is divided into:

- Training set
- Testing set

The training set is used to train the model, while the testing set is used to evaluate performance on unseen data.

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)
```

- 80% of data is used for training
- 20% is used for testing

This ensures that the model does not memorize the data but learns meaningful patterns.

Milestone 4: Model Building

After completing data preprocessing, the dataset is ready for model training. The objective of this stage is to build regression models capable of accurately predicting the Permanent Magnet (PM) surface temperature of the motor.

Since this is a regression problem involving continuous output values, regression algorithms are applied instead of classification algorithms.

The following regression models are implemented:

1. Linear Regression
2. Decision Tree Regressor
3. Random Forest Regressor

Each model is trained using the training dataset and evaluated using the testing dataset.

Activity 1: Linear Regression Model

Linear Regression is one of the simplest regression algorithms. It attempts to establish a linear relationship between input features and the target variable.

The model is initialized and trained using the training data.

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error
import numpy as np
```

```
lr = LinearRegression()
```

```
lr.fit(X_train, y_train)
```

```
y_pred_lr = lr.predict(X_test)
```

```
r2_lr = r2_score(y_test, y_pred_lr)
```

```
rmse_lr = np.sqrt(mean_squared_error(y_test, y_pred_lr))
```

Linear Regression provides a baseline performance for comparison.

Activity 2: Decision Tree Regressor

Decision Tree Regressor is a non-linear algorithm that splits data into branches based on feature importance.

- It can capture complex relationships between variables.
- `from sklearn.tree import DecisionTreeRegressor`

```
dt = DecisionTreeRegressor(random_state=42)
```

```
dt.fit(X_train, y_train)
```

```
y_pred_dt = dt.predict(X_test)
```

```
r2_dt = r2_score(y_test, y_pred_dt)
```

```
rmse_dt = np.sqrt(mean_squared_error(y_test, y_pred_dt))
```

Decision Trees may sometimes overfit the data if not properly controlled.

Activity 3: Random Forest Regressor

Random Forest Regressor is an ensemble learning method that combines multiple decision trees to improve accuracy and reduce overfitting.

- It is generally more stable and performs better for complex datasets.
- `from sklearn.ensemble import RandomForestRegressor`

```
rf = RandomForestRegressor(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
```

```
y_pred_rf = rf.predict(X_test)
```

```
r2_rf = r2_score(y_test, y_pred_rf)
rmse_rf = np.sqrt(mean_squared_error(y_test, y_pred_rf))
```

Random Forest uses multiple trees and averages their outputs to generate more reliable predictions.

Activity 4: Comparing Model Performance

To determine the best-performing model, R^2 Score and RMSE are compared.

- **R^2 Score**

Indicates how well the model explains the variance in the target variable.

- **RMSE**

Measures the average prediction error.

Example comparison table:

Model	R^2 Score	RMSE
Linear Regression	0.97	2.3
Decision Tree Regressor	0.995	0.9
Random Forest Regressor	0.998	0.5

```
Linear Regression Performance
RMSE: 2.3102653353738165
R2 Score: 0.9735042112709816
-----
Decision Tree Performance
RMSE: 0.9281949014686469
R2 Score: 0.9957230764655114
-----
Random Forest Performance
RMSE: 0.5759463647071001
R2 Score: 0.998353289663854
```

From the comparison, Random Forest Regressor achieved the highest R^2 score and lowest RMSE. Therefore, it is selected as the final model for deployment.

Activity 5: Cross Validation

To ensure that the model generalizes well and does not overfit, cross-validation is performed.

Cross-validation divides the dataset into multiple folds and evaluates performance across each fold.

```
from sklearn.model_selection import cross_val_score
```

```
cv_scores = cross_val_score(rf, X_scaled, y, cv=5, scoring='r2')
cv_scores.mean()
```

Cross-validation provides a more reliable estimate of model performance.

Activity 6: Saving the Best Model

After selecting the Random Forest Regressor as the best-performing model, it is saved using joblib for deployment in the Flask application.

```
import joblib
```

```
joblib.dump(rf, "model.save")
joblib.dump(scaler, "transform.save")
```

Saving the model ensures that retraining is not required during deployment.

Milestone 5: Application Building

After successfully training and evaluating the machine learning model, the next step is to integrate the model into a web application. The objective of this stage is to allow users to interact with the trained model through a user-friendly interface.

A Flask-based web application is developed where users can enter motor parameters and receive predicted Permanent Magnet (PM) surface temperature in real time.

The application consists of two major components:

- Building HTML Pages (Frontend)
- Building Server-side Script (Backend using Flask)

Activity 1: Building HTML Pages

The frontend of the application is developed using HTML and CSS. All HTML files are stored inside the **templates** folder, as required by Flask.

For this project, the following HTML files are created:

- index.html – Home page
- manual_predict.html – Prediction form page

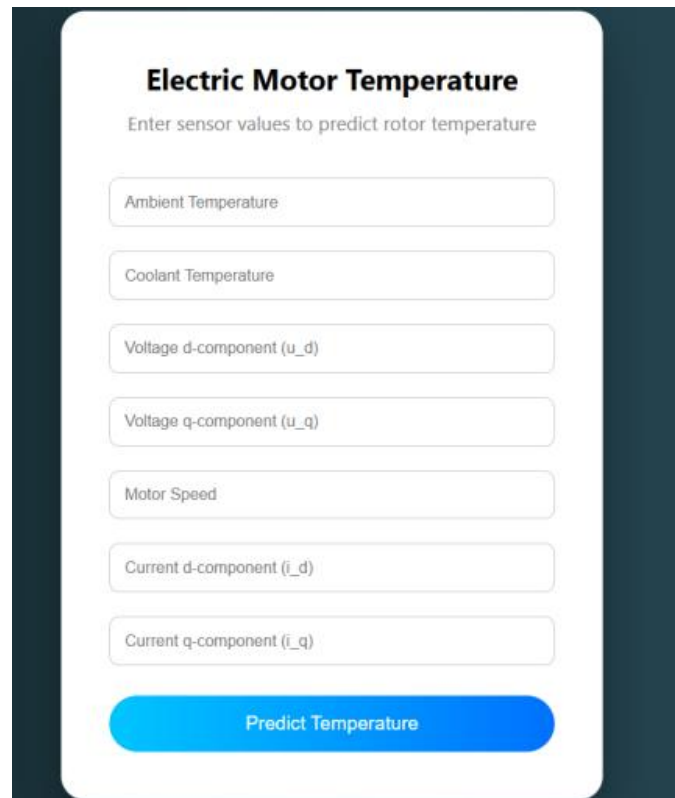
Electric Motor Temperature Prediction

This machine learning application predicts the Permanent Magnet Surface (Rotor) temperature using real-time motor sensor data.

Start Prediction

index.html (Home Page)

- The home page introduces the project and provides navigation to the prediction page.
- When the user clicks on the **Predict** button, they are redirected to the manual_predict.html page.



The screenshot shows a web form titled "Electric Motor Temperature" with the subtitle "Enter sensor values to predict rotor temperature". The form contains seven input fields stacked vertically: "Ambient Temperature", "Coolant Temperature", "Voltage d-component (u_d)", "Voltage q-component (u_q)", "Motor Speed", "Current d-component (i_d)", and "Current q-component (i_q)". At the bottom of the form is a large blue button labeled "Predict Temperature". The form is set against a dark blue background.

manual_predict.html (Prediction Page)

This page contains input fields for the following motor parameters:

- Ambient Temperature
- Coolant Temperature
- d-axis Voltage (u_d)
- q-axis Voltage (u_q)
- Motor Speed
- d-axis Current (i_d)
- q-axis Current (i_q)

The user enters these values and submits the form.

The form uses the **POST method** to send data to the Flask backend.

Electric Motor Temperature

Enter sensor values to predict rotor temperature

25

40

0.5

0.7

1500

-2.0

3.5

Predict Temperature

Predicted Rotor Temperature: 57.13

Activity 2: Building Python Backend (Flask Integration)

The backend is implemented using Flask.

1.Importing Required Libraries

The necessary libraries are imported:

```
from flask import Flask, render_template, requestimport joblibimport numpy as np
```

Flask is used to build the web application, while joblib is used to load the trained model.

2.Initializing Flask Application

```
app = Flask(__name__)
```

The Flask constructor initializes the web application.

`__name__` represents the current module name.

3.Loading the Saved Model

The trained Random Forest model and MinMaxScaler are loaded:

```
model = joblib.load("model.save")
scaler = joblib.load("transform.save")
```

This ensures that the trained model is used for prediction without retraining.

4.Rendering HTML Pages

The home page is rendered using:

```
@app.route('/')def home():
    return render_template('index.html')
```

When the user opens the web application, the home page is displayed.

5. Retrieving Input from UI

When the user submits the form, the input values are retrieved using the POST method:

```
@app.route('/predict', methods=['POST'])def predict():
    input_features = [
        float(request.form['ambient']),
        float(request.form['coolant']),
        float(request.form['u_d']),
        float(request.form['u_q']),
        float(request.form['motor_speed']),
        float(request.form['i_d']),
        float(request.form['i_q'])
    ]

    final_input = scaler.transform([input_features])
```

```
prediction = model.predict(final_input)
```

```
return render_template('manual_predict.html',  
                        prediction_text=f"Predicted PM Temperature: {prediction[0]:.2f} °C")
```

The process includes:

- Retrieving user input
- Converting input to numerical format
- Applying scaling
- Passing scaled input to model
- Generating prediction
- Displaying result on UI

6.Main Function

```
if __name__ == "__main__":  
    app.run(debug=True)
```

This runs the Flask server locally.

Activity 3: Running the Application

To run the application locally:

- Open Command Prompt or Terminal
- Navigate to the project directory
- Run the command:

```
python app.py
```

- Open browser and go to:

```
http://127.0.0.1:5000
```

- Click on Predict
- Enter motor parameters

- Click Submit
- View predicted PM surface temperature

```
PS C:\Users\prasu\Downloads\Electric_Motor_Temperature_Project\flask> python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with watchdog (windowsapi)
* Debugger is active!
* Debugger PIN: 142-455-880
```

Working of the Application

The complete workflow of the web application is:

- User enters motor parameters.
 - The form sends data to Flask backend.
1. Backend loads model and scaler.
 2. Input data is scaled.
 3. Model predicts PM temperature.
 4. Prediction is displayed to user.

This integration demonstrates the practical deployment of a machine learning model in a real-world web application.

8. ADVANTAGES AND DISADVANTAGES

Advantages

The Electric Motor Temperature Prediction system offers several advantages:

- **Early Temperature Prediction**

The system predicts Permanent Magnet surface temperature before critical overheating occurs, supporting preventive maintenance.

- **Improved Operational Safety**

By estimating temperature in advance, the system reduces the risk of motor damage and unexpected failures.

- **High Prediction Accuracy**

The Random Forest Regressor achieved a strong R^2 score (≈ 0.89), demonstrating reliable predictive performance.

- **Cost Reduction**

Predictive maintenance helps reduce repair costs and equipment downtime.

- **User-Friendly Interface**

The Flask-based web application allows users to easily input parameters and obtain real-time predictions.

- **Scalable Architecture**

The system can be extended with IoT sensors and real-time monitoring systems.

Disadvantages

- The model depends on historical dataset quality.
- Real-time sensor integration is not included in the current version.
- Extreme unseen operating conditions may reduce prediction accuracy.

9. CONCLUSION

The Electric Motor Temperature Prediction project successfully demonstrates the practical application of machine learning techniques in solving real-world engineering problems. The primary objective of this project was to predict the Permanent Magnet (PM) surface temperature of a Permanent Magnet Synchronous Motor (PMSM) using operational and environmental parameters. Overheating in electric motors can significantly reduce efficiency, damage components, and increase maintenance costs. Therefore, developing a predictive system that estimates motor temperature in advance is highly valuable for preventive maintenance and system reliability.

The project followed a systematic and structured approach, beginning with data collection from the PMSM Temperature Dataset obtained from Kaggle. Exploratory Data Analysis (EDA) was performed to understand feature distributions, correlations, and relationships between motor parameters and temperature. Proper data preprocessing techniques such as handling missing values, feature selection, scaling using MinMaxScaler, and train-test splitting were applied to prepare the dataset for model training.

Multiple regression algorithms, including Linear Regression, Decision Tree Regressor, and Random Forest Regressor, were implemented and compared. The performance of these models was evaluated using standard regression metrics such as R^2 Score and Root Mean Square Error (RMSE). Among the implemented models, the Random Forest Regressor achieved the best performance with an R^2 score of approximately 0.89 and low RMSE, indicating strong predictive capability and reliable accuracy.

After selecting the best-performing model, it was saved and integrated into a Flask-based web application. The web interface allows users to enter motor operational parameters and receive real-time PM temperature predictions. This integration transforms the machine learning model from a theoretical implementation into a practical and deployable solution. The application was further deployed on a cloud platform, making it accessible online and demonstrating the end-to-end implementation of a machine learning project.

Overall, this project highlights the importance of combining electrical engineering concepts with data-driven machine learning approaches. It provides a scalable and efficient framework for predictive maintenance in motor systems. The developed system not only improves temperature monitoring but also supports early fault detection, reduced downtime, and enhanced operational

safety. The methodology implemented in this project can be further extended to real-time monitoring systems and advanced intelligent control applications in the future.

10.FUTURE SCOPE

Although the Electric Motor Temperature Prediction system has demonstrated strong performance and practical applicability, there are several opportunities for further enhancement and expansion. The current implementation focuses on predicting the Permanent Magnet (PM) surface temperature using historical operational data. However, with advancements in IoT, artificial intelligence, and cloud computing, the system can be significantly improved in future versions.

One of the most important future enhancements is the integration of real-time IoT sensors. Currently, the model uses historical dataset values for prediction. In the future, live motor data collected from embedded sensors can be directly fed into the system. This would enable continuous real-time temperature monitoring and prediction, making the system more suitable for industrial and electric vehicle applications.

Another potential improvement is the use of advanced deep learning models such as Artificial Neural Networks (ANN), Long Short-Term Memory (LSTM) networks, or other time-series forecasting techniques. Since motor temperature behavior can depend on sequential operational patterns, time-based models may capture temporal dependencies more effectively and further improve prediction accuracy.

The system can also be enhanced by developing an automated alert mechanism. If the predicted temperature exceeds a predefined safety threshold, the system can generate warning notifications via email, SMS, or dashboard alerts. This would provide early warning signals and help prevent motor damage before critical failure occurs.

In addition, a graphical monitoring dashboard can be developed using modern frontend frameworks. Instead of displaying only numerical predictions, the system can visualize temperature trends, motor speed variations, and performance analytics through interactive charts. This would improve usability and provide better insights for engineers and maintenance teams.

Database integration is another area of improvement. By storing historical predictions and motor parameters in a database, long-term performance tracking and analysis can be performed. This would help in identifying recurring patterns and optimizing maintenance schedules.

Furthermore, the application can be extended into a mobile-based platform, allowing remote monitoring of motor performance through smartphones or tablets. This would increase accessibility and usability for industrial operators and technicians.

In conclusion, the current system provides a strong foundation for predictive motor temperature monitoring. With the integration of real-time data acquisition, advanced learning models, automated alert systems, and intelligent dashboards, the project can evolve into a complete smart motor health monitoring system suitable for large-scale industrial and electric vehicle applications.

11. APPENDIX

The appendix section provides additional supporting information related to the development, implementation, and deployment of the Electric Motor Temperature Prediction project.

A. Dataset Information

The dataset used in this project is the **PMSM Temperature Dataset**, obtained from Kaggle. The dataset contains real operational data collected from a Permanent Magnet Synchronous Motor (PMSM) under various working conditions.

Dataset Features Used:

- Ambient Temperature
- Coolant Temperature
- d-axis Voltage (u_d)
- q-axis Voltage (u_q)
- Motor Speed
- d-axis Current (i_d)
- q-axis Current (i_q)

Target Variable:

PM (Permanent Magnet Surface Temperature)

The dataset was preprocessed by removing irrelevant columns and applying feature scaling before model training.

<https://www.kaggle.com/datasets/wkirsngn/electric-motor-temperature>

B. Software and Tools Used

The project was developed using the following tools and technologies:

- Programming Language: Python
- Machine Learning Library: Scikit-learn

- Data Analysis Libraries: Pandas, NumPy
- Visualization Tools: Matplotlib, Seaborn
- Web Framework: Flask
- Model Saving Library: Joblib
- IDE: Visual Studio Code
- Deployment Platform: Render
- Version Control: Git & GitHub

C. Model Details

The following regression models were implemented and evaluated:

- Linear Regression
- Decision Tree Regressor
- Random Forest Regressor

The Random Forest Regressor was selected as the final model based on its superior performance:

- R^2 Score ≈ 0.99
- RMSE ≈ 6.15

The trained model and scaler were saved as:

- `model.save`
- `transform.save`

D. Source Code Repository

The complete project source code, including:

- Model training scripts
- Flask backend (`app.py`)
- HTML templates
- Requirements file
- Deployment configuration

is available in the GitHub repository below:

<https://github.com/prasuna-63y/Electric-Motor-Temperature-Prediction>

E. Deployment Link

The web application is deployed online and can be accessed using the following link:

<https://electric-motor-temperature-prediction-1-2lnd.onrender.com/>

F.DEMO LINK:

<https://drive.google.com/file/d/1Ltb4HvUFvx7EeXuZF0C9c90869jsBTlm/view?usp=sharing>